

HTB ACADEMY

JavaScript Deobfuscation

HTB Academy · Module 41 · Malware Analysis / Blue Team Track · Walkthrough

A practical, illustrated guide to identifying, unpacking, and analyzing obfuscated JavaScript — covering every module section with the full deobfuscation workflow, encoding/decoding techniques, and the complete skills assessment. Flags are partially masked below (first/middle/last character shown) in line with HTB's sharing guidance.

Platform	HackTheBox Academy
Module	JavaScript Deobfuscation (#41)
Completion	100% — all sections and questions complete
Analyst	Chayan Panchal · @tron874
Relevance	Malware analysis, incident response, blue-team triage

1 What Is JavaScript Obfuscation?

Obfuscation transforms readable code into a functionally identical but intentionally confusing version. The code still runs exactly the same — it just becomes very hard for a human to read at a glance.

Use case	Why it matters
Hide malware payloads	Conceals C2 callback URLs, download commands, dropper logic from analysts and AV
Evade IDS/IPS detection	Signature-based systems can't match known patterns inside heavily obfuscated code
Protect intellectual property	Legitimate developers obfuscate to slow down reverse engineering
Client-side auth bypass	Hiding API keys/tokens directly in JS — insecure, but still common in the wild

Identification guide — four patterns you'll see in the wild



Minification

```
eval(function(p,a,c,k,e,d){...
```

-> one line, no whitespace. Common in production .min.js



Packing (Dean Edwards)

```
eval(function(p,a,c,k,e,d){ e=function(c){return c.toString(36)}...
```

-> the 6-argument eval wrapper. Signature: (p,a,c,k,e,d). Most common.



obfuscator.io (string array + Base64)

```
var _0x1ec6=["Bg9N","sfrciePH..."]; (function(_0x13249d,_0x1ec6e5){...
```

-> _0x-prefixed variables, string arrays up top, Base64 values. Strong obfuscation.



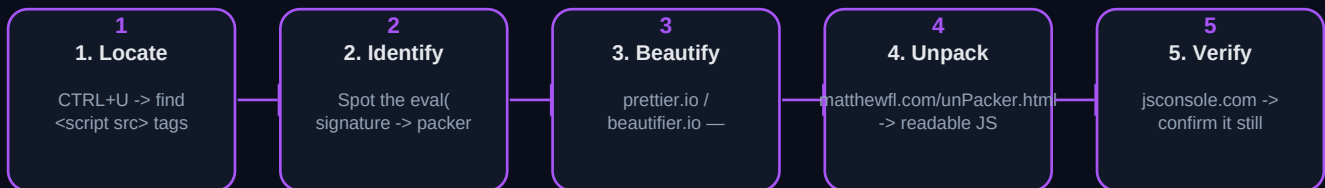
JSFuck

```
[](![]+[])[+[]]+(![]+[])[+[]]+(![]+[])[+[]]+(![]+[])[+[]]+(![]+[])[+[]]...
```

-> only 6 characters used: [] () ! + - verbose, used to bypass filters.

2 Step-by-Step Deobfuscation Workflow

The exact workflow applied against the target web application in this lab.



Result — what the code was hiding

Lab finding: <script src="secret.js"></script>, identified as a Dean Edwards packer. After unpacking:

```
secret.js — deobfuscated output (flag masked)

'use strict';
function generateSerial() {
  var flag = 'HTB{n.....u.....!}';
  var xhr = new XMLHttpRequest();
  var url = '/serial.php';
  xhr.open('POST', url, true);
  xhr.send(null);
};
```

Key findings: (1) a flag variable revealed in plaintext once unpacked; (2) an XMLHttpRequest POSTing to /serial.php; (3) no POST body — the endpoint accepts empty requests.

3 Encoding & Decoding — Base64, Hex, ROT13

The server returns encoded strings after the initial POST request. Recognizing and decoding these by eye is a core triage skill.

Encoding	How to spot it
Base64	Alphanumeric + / and = padding. Length is always a multiple of 4.
Hex	Only 0-9 and a-f, always an even number of characters.
ROT13	Looks like scrambled English — <code>http://www</code> becomes <code>uggc://jjj</code> .



Terminal — Base64 encode / decode

```
$ echo "https://www.hackthebox.eu/" | base64
aHR0cHM6Ly93d3cuaGFja3RoZWJveC5ldS8K

$ echo "aHR0cHM6..." | base64 -d
https://www.hackthebox.eu/

# Lab: decode the server response
$ echo "N2gxNV8xNV9hX3MzY3IzN19tMzU1NGcz" | base64 -d
7.....c.....3 (masked – see notes for full technique)
```



Terminal — Hex encode / decode

```
$ echo "https://www.hackthebox.eu/" | xxd -p
68747470733a2f2f7777772e6861636b746865626f782e65752f0a

$ echo "6874..." | xxd -p -r
https://www.hackthebox.eu/

# Lab: decode the hex key from Skills Assessment Q4
$ echo "4.....3.....e" | xxd -p -r
A.....7.....n
```



Terminal — ROT13 encode / decode

```
$ echo 'https://www.hackthebox.eu/' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
uggcf://jjj.unpxgurobk.rh/

# same command decodes it back – ROT13 is its own inverse
$ echo 'uggcf://jjj.unpxgurobk.rh/' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
https://www.hackthebox.eu/
```

4 Replicating Hidden Functions with curl

Once the JS is deobfuscated and the endpoint identified, the next step is replicating the request by hand with curl — this reveals server-side behavior the front end never shows you.

```
curl — full request chain (flags masked)

$ curl http://TARGET/
<!DOCTYPE html><head><title>Secret Serial Generator</title>...

$ curl -s http://TARGET/serial.php -X POST
7.....c.....3 (Base64-encoded response, masked)

$ curl -s http://TARGET/serial.php -X POST -d "serial=<decoded value>"
HTB{j.....r.....l}

$ curl -s http://TARGET/keys.php -X POST -d "key=<decoded value>"
HTB{r.....m.....B}
```

5 Skills Assessment — Full Solution

The assessment chains every technique from the module into one multi-stage challenge against a live target.

Q1 — Identify the JavaScript file

Inspected HTML source of the target, found an externally loaded script.

```
view-source:http://TARGET  
-> <script src="api.min.js"></script>
```

Flag / Answer api.min.js

Q2 — Run the obfuscated code

Pasted api.min.js into jsconsole.com and executed it — console.log printed a flag.

```
jsconsole.com -> paste & execute api.min.js
```

Flag / Answer HTB{j.....m.....y}

Q3 — Deobfuscate and extract the flag variable

Used UnPacker to unpack the eval wrapper, revealing a flag variable in plaintext.

```
unpacker -> deobfuscate api.min.js  
var flag = 'HTB{n.....u.....!}';  
xhr.open('POST', '/keys.php', true);
```

Flag / Answer HTB{n.....u.....!}

Q4 — Replicate the POST request, get the secret key

Code analysis showed a POST to /keys.php with no body — replicated with curl.

```
$ curl -s http://TARGET/keys.php -X POST  
4.....3.....e
```

Flag / Answer 4.....3.....e

Q5 — Decode the hex key, get the final flag

Identified the response as hex, decoded with xxd, sent it back.



Terminal / Browser

```
$ echo "4.....3.....e" | xxd -p -r
A.....7.....n
$ curl -s http://TARGET/keys.php -X POST -d "key=A.....7.....n"
HTB{r.....m.....B}
```

Flag / Answer

HTB{r.....m.....B}

6 Key Takeaways for Blue Team Analysts

Takeaway	Why it matters
Recognise packer signatures	<code>eval(function(p,a,c,k,e,d){...})</code> = Dean Edwards Packer -> run through UnPacker first
<code>_0x</code> variables = obfuscator.io	Variable names starting with <code>_0x</code> reliably indicate obfuscator.io + string arrays
Spot Base64 by padding	<code>=</code> or <code>==</code> at the end, alphanumeric content -> decode with <code>base64 -d</code>
Spot hex by character set	Only 0-9 and a-f -> decode with <code>xxd -p -r</code>
ROT13 keeps English shape	<code>http://www -> uggc://jjj</code> is recognisable -> decode with <code>tr</code>
Always verify with jsconsole	After deobfuscating, re-run in <code>jsconsole.com</code> to confirm it still behaves the same
Replicate JS requests with curl	Any XHR/fetch call can be replicated manually -> reveals hidden server endpoints
Multi-stage chains are common	Real malware: obfuscation -> encoding -> POST -> encoded response -> decode -> next stage

Quick Reference — Every Command Used in This Module

Grouped by purpose — use this page as a cheat-sheet during lab work.



Source inspection

CTRL+U	View raw HTML source
CTRL+SHIFT+Z	Open Firefox Debugger (DevTools)
{ } button (DevTools)	Pretty-print / beautify minified JS



Deobfuscation tools (online)

prettier.io/playground	Format and indent obfuscated JS
beautifier.io	Alternative JS/HTML/CSS beautifier
matthewfl.com/unPacker.html	Unpack Dean Edwards eval(p,a,c,k,e,d)
jsconsole.com	Run JS snippets, verify output
obfuscator.io	Obfuscate JS — understand it to reverse it



Encoding / decoding

echo "TEXT" base64	# Base64 encode
echo "B64" base64 -d	# Base64 decode
echo "TEXT" xxd -p	# Hex encode
echo "HEX" xxd -p -r	# Hex decode
echo 'TEXT' tr 'A-Za-z' 'N-ZA-Mn-za-m'	# ROT13 (same cmd both ways)



HTTP requests with curl

curl http://TARGET/	# basic GET
curl -s http://TARGET/	# silent mode
curl -X POST http://TARGET/serial.php	# empty POST
curl -X POST URL -d "k=v"	# POST with form data
curl -v http://TARGET/	# verbose — see headers